

Aim

Write a program to implement a Recursive Descent Parser for arithmetic expressions.

Algorithm

1. Start the program.
2. Read the input arithmetic expression as a string.
3. Initialize index `i = 0` and error flag `error = 0`.
4. Use the following grammar for parsing:

$$\begin{aligned} E &\rightarrow T E' \\ E' &\rightarrow + T E' \mid \\ T &\rightarrow F T' \\ T' &\rightarrow * F T' \mid \\ F &\rightarrow (E) \mid \text{id} \end{aligned}$$

5. Define recursive functions for each non-terminal:
 - `E()` calls `T()` followed by `Eprime()`.
 - `Eprime()` checks for `'+'` and continues recursively.
 - `T()` calls `F()` followed by `Tprime()`.
 - `Tprime()` checks for `'*'` and continues recursively.
 - `F()` checks for parentheses or identifiers (alphanumeric).
6. At the end, if the entire input is parsed and `error == 0`, accept the input. Otherwise, reject it.
7. Stop.

Program Code

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

char input[100];
int i = 0, error = 0;
```

```

void E();
void Eprime();
void T();
void Tprime();
void F();

void E() {
    T();
    Eprime();
}

void Eprime() {
    if (input[i] == '+') {
        i++;
        T();
        Eprime();
    }
}

void T() {
    F();
    Tprime();
}

void Tprime() {
    if (input[i] == '*') {
        i++;
        F();
        Tprime();
    }
}

void F() {
    if (input[i] == '(') {
        i++;
        E();
        if (input[i] == ')') {
            i++;
        } else {
            error = 1;
        }
    } else if (isalnum(input[i])) {
        i++;
    } else {
        error = 1;
    }
}

```

```

    }
}

int main() {
    printf("Enter an arithmetic expression: ");
    scanf("%s", input);

    E();

    if (input[i] == '\0' && error == 0) {
        printf("Accepted.\n");
    } else {
        printf("Rejected.\n");
    }

    return 0;
}

```

Sample Input and Output

Input 1

i+i*i

Output 1

Accepted.

Input 2

(i+i

Output 2

Rejected.